Project 1: Secure Software Design

Sindhuja Martha

Student ID: 005035378

Secure Software Development (MSCS-535-B01): **Chapters 1-4**

MS in Computer Science, University of the Cumberlands, KY

Professor: **Dr. Yousef Nijim**

March 15, 2026

Secure Database Access Over HTTPS with Protection Against SQL Injection

Introduction

In modern organizations, web applications frequently interact with databases to store and retrieve important information such as employee accounts, authentication credentials, and operational data. Because these systems are often accessible over networks, they become common targets for cyberattacks. Two common threats are social engineering attacks, such as phishing emails, and technical attacks such as SQL injection. A phishing email may trick employees into revealing login credentials, while a SQL injection attack may allow attackers to manipulate database queries to gain unauthorized access to sensitive data (Kurose, 2021).

The purpose of this assignment is to demonstrate how secure coding practices can protect a database-driven web application from these threats. Specifically, the project implements a web application that ensures database access occurs through an encrypted HTTPS connection and that user input is handled safely to prevent SQL injection attacks.

Background

Web applications rely on databases to store information like user accounts and system records. When user input is not checked carefully, attackers can enter harmful SQL commands into fields such as login forms. This type of attack, called SQL injection, lets attackers get around authentication, view private data, or change stored information.

Keeping data safe while it travels over a network is also very important. If the connection between a user and a web application is not secure, attackers on the same network can steal sensitive details like usernames and passwords. Using HTTPS encrypts the data sent between the client and server, so only authorized parties can read or change it (Ross, 2021).

Implementation Overview

This project uses the Python Flask framework, which is lightweight and flexible for building web apps. It lets users register and log in, and it connects securely to a SQLite database. The server runs over HTTPS, so all communication between users and the app is encrypted. The app creates a self-signed TLS certificate with an open-source cryptography library. This lets the server set up an HTTPS connection locally. While self-signed certificates cause browser warnings, they are often used in development and academic settings to show secure network communication.

How Secure Database Access Is Implemented

The application ensures secure database access through the use of parameterized SQL queries. Rather than putting user input straight into SQL statements, it uses placeholders and sends the input as parameters. This stops the database from treating user input as SQL commands. Because of this, SQL injection attacks do not work since any harmful input is handled as regular data.red in plain text. Instead, passwords are processed using secure hashing functions before being stored in the database. This practice protects user credentials even if the database is compromised.

Security Mechanisms Implemented

Several security techniques were incorporated into the system. First, HTTPS encryption ensures that all communication between the client and the web server is protected while traveling across the network. Second, parameterized queries prevent SQL injection by separating user input from SQL command execution. Third, password hashing protects stored credentials from being exposed in readable form. Together, these mechanisms create multiple layers of protection for the application.

Code Snippets:

Auth.py

```
from sqlite3 import IntegrityError
from .db import get_db
from .security import hash_password, verify_password, validate_username, validate_password

def register_user(username: str, password: str):
    # Check the username before saving it.
    if not validate_username(username):
        return False, "Username must be 3-30 characters and contain only letters, numbers, _, ., or -."
    # Check the password rules before saving it.
    if not validate_password(password):
        return False, "Password must be at least 8 characters and include uppercase, lowercase, and a number."

    db = get_db()
    try:
        # Use placeholders so user input is treated as data, not SQL code.
        db.execute(
            "INSERT INTO users (username, password_hash) VALUES (?, ?)",
            (username, hash_password(password))
        )
        db.commit()
        return True, "User registered securely."
    except IntegrityError:
        # This happens when the username is already in the table.
        return False, "Username already exists."

def authenticate_user(username: str, password: str):
    db = get_db()
    # Look up the user with a parameterized query.
    user = db.execute(
        "SELECT id, username, password_hash FROM users WHERE username = ?",
        (username,)
    ).fetchone()

    # Compare the entered password with the saved password hash.
    if user and verify_password(user["password_hash"], password):
        return True, {"id": user["id"], "username": user["username"]}
    return False, None

def fetch_user(username: str):
    db = get_db()
    # Return basic user data without exposing the password hash.
    user = db.execute(
        "SELECT id, username FROM users WHERE username = ?",
        (username,)
    ).fetchone()
    return dict(user) if user else None
```

Routes.py

```
from flask import jsonify, request
from .auth import register_user, authenticate_user, fetch_user

def register_routes(app):
    @app.get("/")
    def home():
        # Simple health check route.
        return jsonify({
            "message": "SecureAccess is running over HTTPS.",
            "endpoints": ["/register", "/login", "/user/<username>"]
        })

    @app.post("/register")
    def register():
        # Read JSON safely, even if the request body is missing or invalid.
        data = request.get_json(silent=True) or {}
        username = data.get("username", "").strip()
        password = data.get("password", "")

        success, message = register_user(username, password)
        status = 201 if success else 400
        return jsonify({"success": success, "message": message}), status

    @app.post("/login")
    def login():
        # Read the login details sent by the client.
        data = request.get_json(silent=True) or {}
        username = data.get("username", "").strip()
        password = data.get("password", "")

        success, user = authenticate_user(username, password)
        if success:
            return jsonify({"success": True, "message": "Login successful.", "user": user})
        return jsonify({"success": False, "message": "Invalid credentials."}), 401

    @app.get("/user/<username>")
    def get_user(username):
        # Look up a user by username.
        user = fetch_user(username)
        if user:
            return jsonify({"success": True, "user": user})
        return jsonify({"success": False, "message": "User not found."}), 404
```

Execution Results

```
REGISTER: 400 {"message":"Username already exists.","success":false}

LOGIN: 200 {"message":"Login successful.","success":true,"user":{"id":1,"username":"alice"}}

SQL INJECTION ATTEMPT: 401 {"message":"Invalid credentials.","success":false}
```

Figure above shows the terminal output in VS Code when an existing user attempts to log in and a SQL injection attack is also tested.

```
Status Code: 401
Response: {"message":"Invalid credentials.","success":false}

PASS: SQL injection attempt failed as expected.
```

Figure above shows the terminal output in VS Code when a user attempts a SQL injection attack

Discussion

The results show that the application works as expected. User registration and login connect to the database and keep communication secure with HTTPS. In testing, normal logins worked, and SQL injection attempts were blocked. This shows that using parameterized queries helps stop attackers from changing database commands.

This experiment shows why it is important to use several security measures when building web applications. Network encryption keeps data safe while it is sent, and secure coding protects the database from harmful input.

Conclusion

This project demonstrates how a web application can securely access a database while protecting against common cybersecurity threats. By implementing HTTPS communication, parameterized SQL queries, and secure password storage, the system reduces the risk of credential theft and database compromise. These techniques represent essential best practices for building secure and reliable web applications in modern computing environments.

References

Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing* (RFC 7230). Internet Engineering Task Force.

Kurose, J. F., & Ross, K. W. (2021). *Computer networking: A top-down approach* (8th ed.). Pearson.

NIST. (2022). *Digital identity guidelines*. National Institute of Standards and Technology.

Git hub link: https://github.com/sindhuja90comp/MSCS535_A5_SecureAccess.git